Open Source Agentic AI Architecture

A cost-effective alternative to platform AI agents

The disciplined open source path produces a defensible institutional posture at an order-of-magnitude lower cost than frontier-default managed alternatives (i.e. Claude), and at materially better auditability than any vendor attestation can replicate. The right answer is context-dependent — but the option set is wider than most institutions realize.

01 · THE REQUEST FLOW

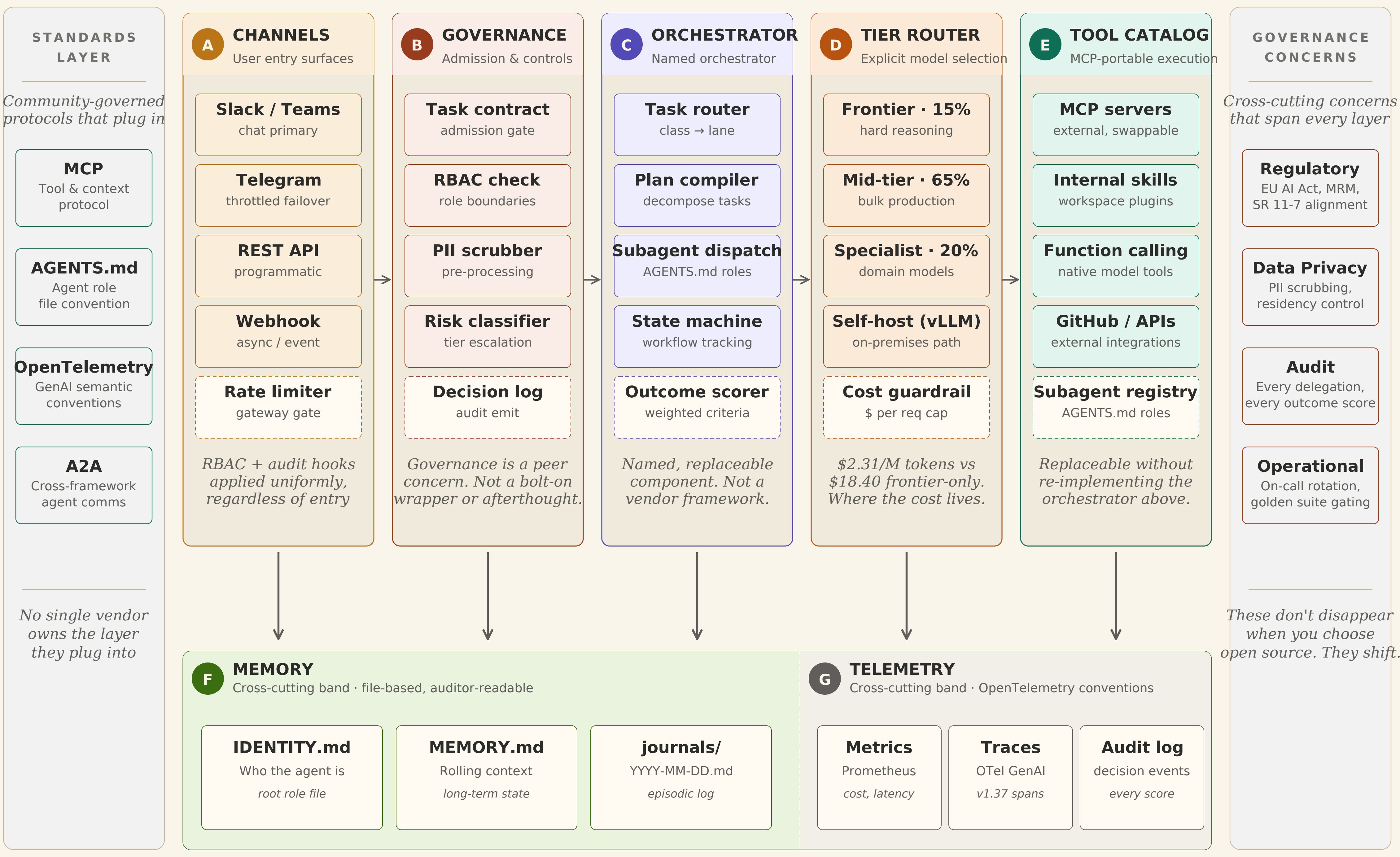
Every request walks the same path. *No exceptions.*

1. **INGRESS** — Where the request enters the system ↓ A
2. **AUTHORIZE** — What gates and permissions apply ↓ B
3. **PLAN** — How work is decomposed ↓ C
4. **ROUTE** — Which model tier serves the request ↓ D
5. **EXECUTE** — What tools and skills actually fire ↓ E
6. **PERSIST** — What state survives across requests ↓ F
7. **OBSERVE** — What gets traced and audited ↓ G

Each step above maps 1:1 to an architectural layer below. Badge colors and letters connect them.

02 · THE ARCHITECTURE

Five layers, two cross-cutting bands. *Every component is replaceable at the protocol layer.*



03 · DESIGN DECISIONS

Why each layer is structured the way it is. *The rationale behind the architecture.*

1. Why a named orchestrator? *Column C*
Frameworks (LangGraph, CrewAI) lock you into their abstractions. A named orchestrator is yours.

2. Why file-based memory? *Band F*
Vector DBs are managed services with their own lock-in. Plain files are grep-able, git-able, auditor-readable.

3. Why explicit tier routing? *Column D*
80% of agent requests don't need frontier reasoning. Explicit selection captures most of the cost savings.

4. Why MCP for tools? *Column E*
Native function calls bind tools to one model. MCP makes them portable across providers and entrants.

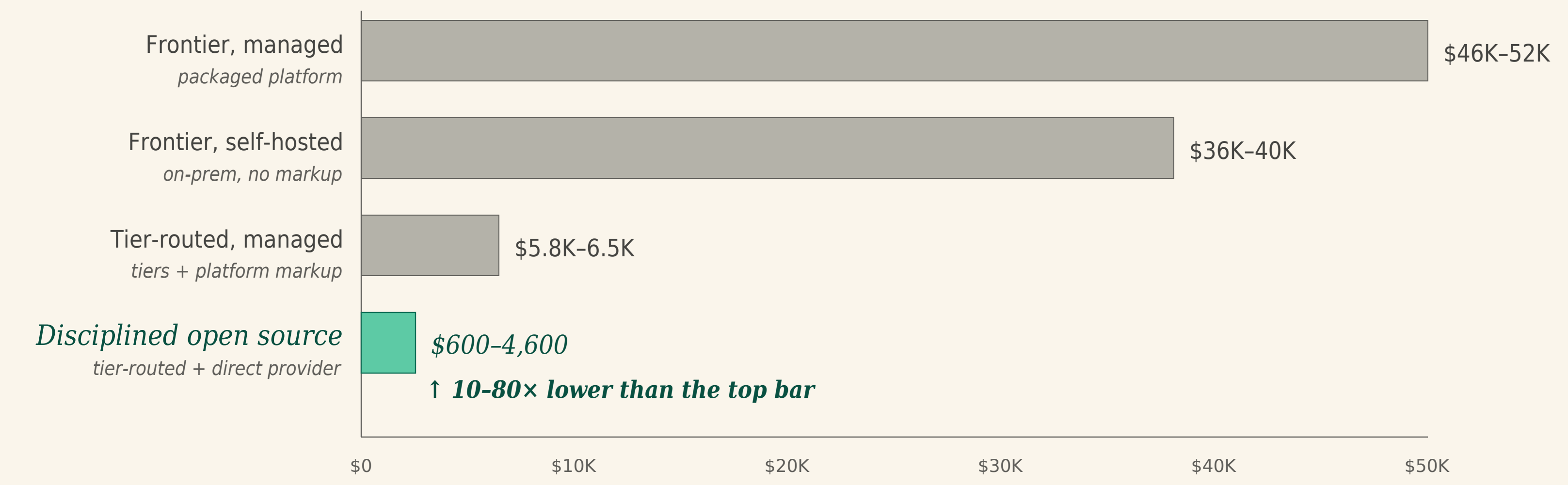
5. Why governance at admission? *Column B*
Bolt-on guardrails get bypassed at scale. Governance at the gate catches risk before the orchestrator commits.

6. Why five layers plus two bands? *Structural shape*
Three hides too much. Twelve invites optimization. Five-plus-two matches the interfaces teams can own.

04 · THE COST EVIDENCE

10–80× lower cost at scale. *Structural, not transient.*

Monthly operating cost at 10,000 agent requests · 2026 reference data · institutional workload mix



MECHANISM 01
Tier routing
80% skip the frontier tier.

MECHANISM 02
No platform markup
Direct provider or self-host. 25–40% margin gone.

MECHANISM 03
Vendor-neutral
Swap providers as costs shift.

05 · WHAT MAKES IT INSTITUTIONALLY VIABLE

Six **discipline** conditions separate *viable* open source from *hopeful* open source.

1. Cost discipline *10–80× lower at scale*
Tier routing + no platform markup. The differential is structural, not a discount that will close.

2. Audit transparency *No vendor opacity*
Plain files, OTel-standard traces, decision events in a log. Auditors can read what happened.

3. Migration optionality *Configuration, not project*
Standards-layer integration means swapping the orchestrator or model provider is a config change.

4. Standards governance *Linux Foundation, not a vendor*
MCP donated Dec 2025. AGENTS.md at 60K+ projects. Community ownership replaces vendor lock-in.

5. Operational ownership *Small dedicated team*
On-call rotation, observability as primary surface, golden suite as operational contract. Day one.

6. Strategic horizon *Multi-year, not multi-quarter*
If the question is “what works in 90 days,” the honest answer is packaged. DOS rewards patience.

You don't need a packaged platform to build production agentic AI.
The disciplined alternative is cheaper, more transparent, and yours to own.

Derived from "Building Agentic AI Systems with Open Source Tools: An End-to-End Architecture Case Study" · Barry Eisenberg, NextFi Advisors · May 2026